

Department of Computer and Software Engineering-ITU

CE203L: Computer Organization and Architecture Lab

Course Instructor: Mr. Aftab Alam	Dated:
Lab Engineer: Muhammad Usama Riaz	Semester: 4th
Session: 2024-2028	Batch: BSCE24

Lab 8. Implementing the Single Cycle RISC-V data-path components in Verilog

Name	Roll No	Obtained Marks/35

Checked on: _____

Signature: _____

Objective

The goal of this handout is to design and implement different components required to make the MIPS data path.

Equipment and Component

Component Description	Value	Quantity
Computer/Xilinx Vivado	On Campus	1

Conduct of Lab

1. Students are required to perform this experiment individually.
2. In case the lab experiment is not understood, the students are advised to seek help from the course instructor, lab engineers, assigned teaching assistants (TA) and lab attendants.

Theory and Background

Processor Architecture Overview

A processor consists of two cooperating units: the Datapath and the Control Unit. The datapath contains the functional elements that hold and transform data — registers, memories, the ALU, and interconnecting buses. The control unit generates the signals that direct the datapath, telling it which operations to perform and which data paths to activate for a given instruction.

Single-Cycle Design Philosophy

In a single-cycle processor, every instruction — regardless of type — completes in exactly one clock cycle. The clock period must therefore be long enough to accommodate the longest possible instruction path (typically a load instruction: Instruction Fetch → Register Read → ALU Compute → Data Memory Read → Write-Back). While this design is straightforward to implement and reason about, it is inherently inefficient because fast instructions are slowed down to match the slowest one.

RISC-V Instruction Formats

The RV32I base integer ISA encodes instructions in five formats depending on the operand layout. The following table summarises the bit-field assignments for the formats implemented in this lab:

Bits [31:25]	Bits [24:20]	Bits [19:15]	Bits [14:12]	Bits [11:7]	Bits [6:0]	Type
imm[11:5]	rs2	rs1	funct3	imm[4:0]	0100011	S-type (Store)
imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	1100011	B-type (Branch)
funct7	rs2	rs1	funct3	rd	0110011	R-type (ALU)
imm[11:0]	–	rs1	funct3	rd	0010011	I-type (ALU Imm)
imm[11:0]	–	rs1	funct3	rd	0000011	I-type (Load)

Key fields:

- rs1, rs2 — source register indices (5-bit, selecting from x0–x31).
- rd — destination register index (5-bit).
- funct3, funct7 — encode the specific operation within an instruction class.
- imm — immediate value; always sign-extended to 32 bits by the Immediate Generator.

Datapath Signal Flow

The diagram below traces how signals flow through the single-cycle datapath for a typical R-type instruction (e.g., add rd, rs1, rs2):

PC → Instruction Memory → [rs1, rs2, rd, funct3, funct7]

Register File (read rs1, rs2) → ALU → Register File (write rd)

PC + 4 → PC (next instruction)

For a Load instruction (lw rd, imm(rs1)):

PC → Instruction Memory → ImmGen(imm) + Register File(rs1) → ALU(ADD) → Data Memory(read) → Register File(write rd) → PC+4

For a Branch instruction (beq rs1, rs2, offset):

PC → Instruction Memory → Register File(rs1, rs2) → ALU(SUB) → zero? → PC + ShiftLeft1(ImmGen(offset)) OR PC+4

ALU Control Encoding

The ALU accepts a 3-bit control signal that selects the operation to perform. The mapping is:

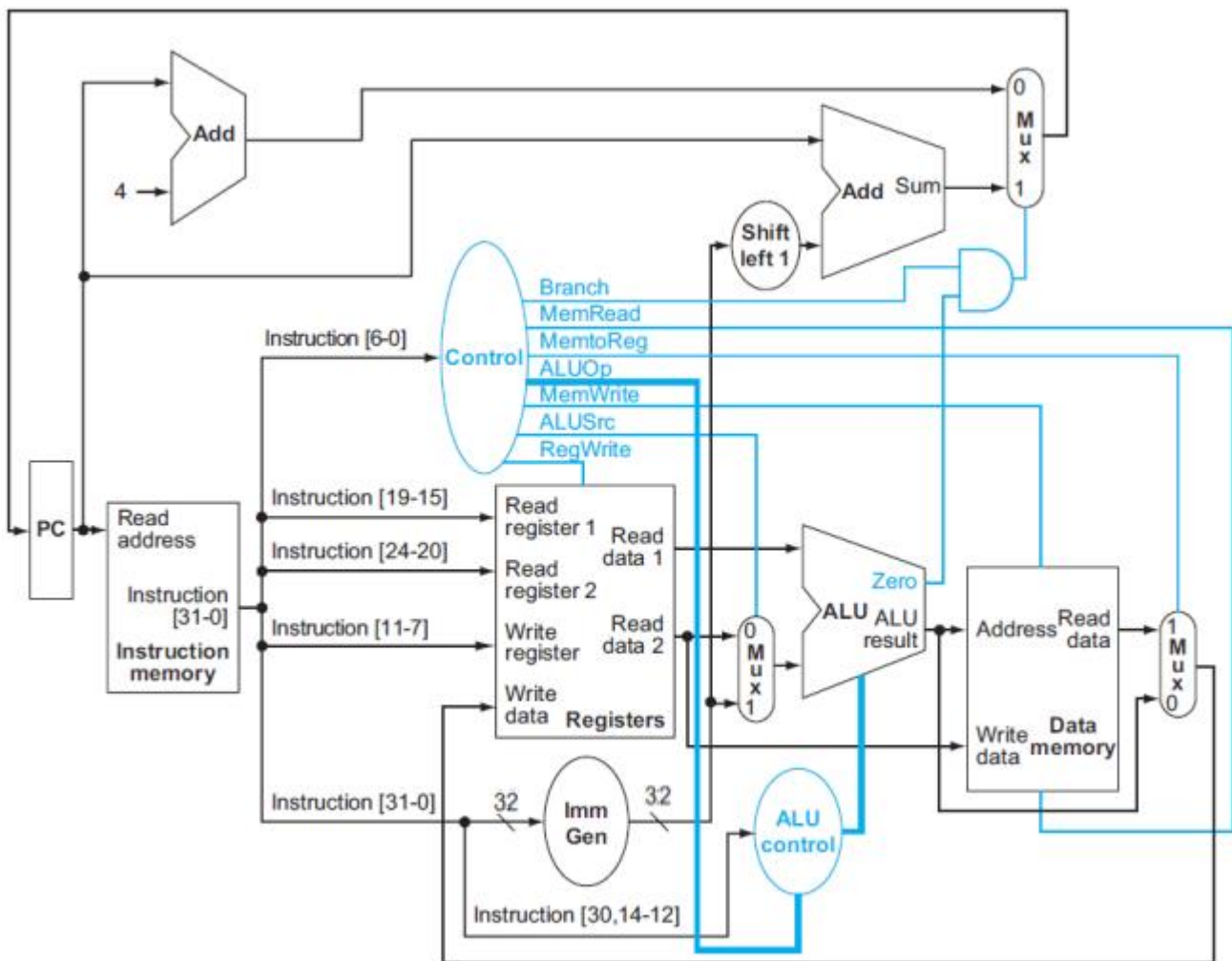
ALUControl [2:0]	Operation	Used By	Description
000	ADD	lw, sw, addi	ADD result; also computes branch targets
001	SUB	beq, bne, sub	Subtraction; zero flag used for branch
010	AND	and, andi	Bitwise AND
011	OR	or, ori	Bitwise OR
100	SLT	slt, slti	Set if less than (signed)
101	XOR	xor, xori	Bitwise XOR

Component Specifications

The following table provides the complete port specifications for each of the 10 datapath components you must implement. Port widths and naming conventions must be followed precisely so that all modules interconnect correctly in the top-level processor.

Component	Inputs	Outputs	Notes
MUX (2:1, 32-bit)	in0, in1 [31:0] sel	out [31:0]	Select between two 32-bit values
MUX (2:1, 5-bit)	in0, in1 [4:0] sel	out [4:0]	Used for register address selection
Adder (32-bit)	a, b [31:0]	sum [31:0]	PC+4 and branch target computation
PC Register	clk, reset pc_next [31:0]	pc [31:0]	Holds current program counter value
Instruction Memory	addr [31:0]	instr [31:0]	ROM; 64-word depth minimum

Register File	clk, RegWrite rs1, rs2, rd [4:0] write_data [31:0]	rd1, rd2 [31:0]	32 registers × 32-bit; x0 always 0
ALU (32-bit)	a, b [31:0] ALUControl [2:0]	result [31:0] zero [0]	ADD, SUB, AND, OR, SLT, XOR
Data Memory	clk, MemRead MemWrite addr [31:0] write_data [31:0]	read_data [31:0]	Word-addressable; 64-word depth
ImmGen	instr [31:0]	imm_ext [31:0]	Sign-extend 12-bit immediate to 32-bit
ShiftLeft1	in [31:0]	out [31:0]	Shifts left by 1 (×2) for branch offset



Single Cycle RISC-V Processor

Lab Tasks

Task:

Design the following components required for the RISC-V data-path:

-Multiplexers (32bit,5bit)

-Adder (32bit)

-PC (32 bit)

-Instruction Memory (32 bit)

-Register File (32 bit)

-ALU (32 bit)

-Data Memory (32 bit)

-Immediate Generation (32bit-32bit)

(Extracts the 12 bit immediate value from 32 bit input instruction and output the sign extended 32 bit value.)

-ShiftLeft (32bit-32bit)

(Takes 32 bit field as input and shift left it 1 times to make it divisible by 2)

Assessment Rubric for Lab

Performance metric	CLO	Able to complete the task over 80% (4-5)	Able to complete the task 50-80% (2-3)	Able to complete the task below 50% (0-1)	Marks
1. Realization of experiment	1	Executes without errors excellent user prompts, good use of symbols, spacing in output. Through testing has been completed.	Executes without errors, user prompts are understandable, minimum use of symbols or spacing in output. Some testing has been completed.	Does not execute due to syntax errors, runtime errors, user prompts are misleading or non-existent. No testing has been completed.	
2. Conducting experiment	1	Able to make changes and answered all questions.	Partially able to make changes and few incorrect answers.	Unable to make changes and answer all questions.	
3. Computer use	2	Document submission timely.	Document submission late.	Document submission not done.	
4. Teamwork	3	Actively engages and cooperates with other group member(s) in effective manner.	Cooperates with other group member(s) in a reasonable manner but conduct can be improved.	Distracts or discourages other group members from conducting the experiment	
5. Laboratory safety and disciplinary rules	3	Code comments are added and does help the reader to understand the code.	Code comments are added and does not help the reader to understand the code.	Code comments are not added.	
6. Data collection	3	Excellent use of white space, creatively organized work, excellent use of variables and constants, correct identifiers for constants, No line-wrap.	Includes name, and assignment, white space makes the program fairly easy to read. Title, organized work, good use of variables.	Poor use of white space (indentation, blank lines) making code hard to read, disorganized and messy.	
7. Data analysis	4	Solution is efficient, easy to understand, and maintain.	A logical solution that is easy to follow but it is not the most efficient.	A difficult and inefficient solution.	
Total (out of 35):					